LÖSUNGEN · LEHREREXEMPLAR

W0·2 – Die Werkstatt: Lösungen

Zum Arbeitsblatt `AQ_W0_AB2_Werkstatt_Basics.docx` · IDE-Level 0.2: siehe `AQ_W0_Akademie_LSG.txt` (Klasse Werkstatt) · alle Werte mit Java (javac 21) nachgerechnet, Compiler-Urteile mit javac geprüft

Aufgabe 0 – Blitz-Warm-up

Ausdruck	`23 / 4`	`23 % 4`	`4 % 23`	`-9 / 2`	`-9 % 2`	`10 - 3 * 2 % 4`
Wert	5	3	4	-4	-1	8

Ausdruck	`(double) 9 / 2`	`(double) (9 / 2)`	`(int) 7.9`	`1 / 2 * 4.0`	`7 > 3 && 2 > 5`	`!(3 == 3) || 4 != 4`
Wert	4.5	4.0	7	0.0	false	false

Gesprächsanlässe: `-9 / 2` rundet **zur 0 hin** (−4, nicht −5); `-9 % 2` ist −1. Bei `10 - 3 * 2 % 4` gilt Punkt vor Strich, `*` und `%` von links: 6 % 4 = 2, 10 − 2 = 8. Bei `1 / 2 * 4.0` wird zuerst ganzzahlig 1 / 2 = 0 gerechnet.

Aufgabe 1 – Der Typ-Detektiv (7 BE)

Nr.	Anweisung	kompiliert?	Wert bzw. Grund
1	`int a = 7.0;`	nein	Kommazahl passt nicht ohne Cast in eine Ganzzahl (möglicher Verlust)
2	`double d = 7;`	ja	7.0 (automatische Umwandlung)
3	`char c = "x";`	nein	"x" ist eine Zeichenkette, kein Zeichen ('x')
4	`String s = 'x';`	nein	'x' ist ein Zeichen, keine Zeichenkette
5	`boolean b = 0;`	nein	In Java ist 0 kein Wahrheitswert
6	`int n = (int) 7.9;`	ja	7 (abgeschnitten)
7	`double q = 7 / 2;`	ja	3.0 (erst ganzzahlig 3, dann umgewandelt)
8	`int m = 10 / 4.0;`	nein	10 / 4.0 ist eine Kommazahl (2.5)
9	`String t = "Gold: " + 5 + 3;`	ja	"Gold: 53" (von links: Zeichenkette + 5, dann + 3)
10	`String u = 5 + 3 + " Gold";`	ja	"8 Gold" (erst 5 + 3 = 8 addiert)
11	`int k = (int) 'A' + 2;`	ja	67 (ASCII-Wert von 'A' ist 65)
12	`boolean g = 3 < 5 < 7;`	nein	3 < 5 ist true; true < 7 ist nicht definiert. Richtig: `3 < 5 && 5 < 7`

Bewertung: a) 3 BE (je 4 richtige Urteile 1 BE), b) 4 BE (je 3 Zeilen 1 BE).

Aufgabe 2 – Wechselgeld (8 BE)

Zeile	betrag	gold	silber	kupfer
1	487			
2		4		
3	87			
4			8	
5				7

b) Nach dem Vertauschen rechnet Zeile 3 mit dem **ganzen** Betrag: `silber` = 487 / 10 = **48** statt 8. gold (4) und kupfer (7) bleiben gleich (2 BE).

c) schnitt = **16.0** (50 / 3 wird ganzzahlig zu 16, dann umgewandelt) · schnitt2 = **16.666…** · gerundet = **17** (16.666… + 0.5 = 17.166… → 17) · abgeschnitten = **16** (3 BE).

d) 49 / 3 = 16.333…; + 0.5 = 16.833… → (int) → **16** (1 BE).

Aufgabe 3 – Die Würfel der Werkstatt (7 BE)

Ausdruck	kleinster Wert	größter Wert
`(int) (Math.random() * 6)`	0	5
`(int) (Math.random() * 6) + 1`	1	6
`(int) Math.random() * 6 + 1`	1	1 (immer 1!)
`(int) (Math.random() * 11) - 5`	-5	5
`(int) (Math.random() * 20) + 10`	10	29

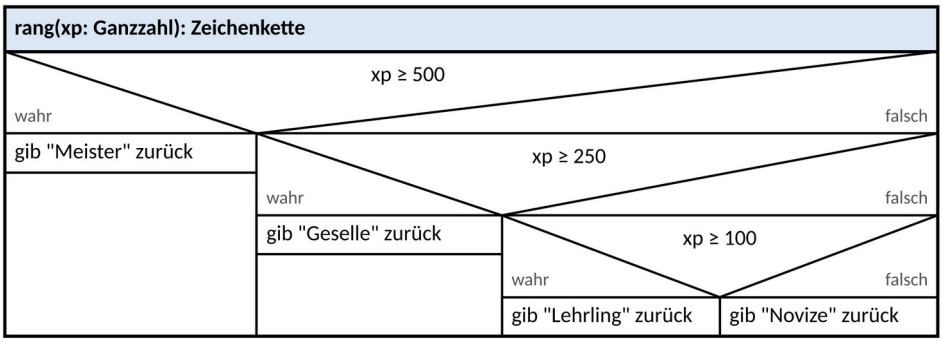
Empirisch geprüft mit je 200 000 Würfen. **b)** `(int) (Math.random() * 7) - 3` – 7 mögliche Werte (−3 … 3), Verschiebung um −3 (2 BE). **c)** Falsch: Bei zwei Würfeln gibt es 36 gleich wahrscheinliche Paare; die Summe 7 entsteht auf 6 Arten (Wahrscheinlichkeit 6/36), die 2 und die 12 nur auf je eine Art (1/36). Bei einer Zufallszahl von 2 bis 12 hätte jede Summe 1/11. Die Werte sind also gleich, die **Verteilung** nicht (2 BE).

Aufgabe 4 – Rangabzeichen (10 BE)

Aufruf	`rang(50)`	`rang(120)`	`rang(300)`	`rang(800)`
Rückgabe	Novize	Lehrling	Lehrling	Lehrling

b) Jeder Wert ab 250 ist auch ≥ 100. Die erste Bedingung `xp >= 100` trifft also schon zu; in einer else-if-Kette werden die weiteren Zweige dann **nicht mehr geprüft**. Die Zweige für Geselle und Meister sind nur erreichbar, wenn xp < 100 ist – dann ist aber auch xp < 250 (2 BE).

c) Korrektur: Bedingungen absteigend prüfen (oder Bereiche mit && angeben). Struktogramm (4 BE: Kopf, Schachtelung im Falsch-Zweig, Bedingungen, Rückgaben):



d) Ja: rang(300) = Geselle, rang(800) = Meister. Ohne else werden **alle** Bedingungen geprüft; jede zutreffende überschreibt r, die letzte zutreffende (die höchste Schwelle) gewinnt. Für alle xp von −10 bis 1999 mit der Korrektur aus c) verglichen: identisch (2 BE).

Aufgabe 5 – Die Zauberfolge (7 BE)

Durchlauf	n % 2 == 0	n	s
vor der Schleife	–	6	0
1	w	3	1
2	f	10	2
3	w	5	3

Durchlauf	n % 2 == 0	n	s
4	f	16	4
5	w	8	5
6	w	4	6
7	w	2	7
8	w	1	8

Rückgabewert **8** (3 BE).

b) Startwert **n = 1**: Die while-Fassung prüft zuerst und liefert **0**; die do-while-Fassung führt den Rumpf einmal aus (1 → 4 → 2 → 1) und liefert **3**. Für alle n von 2 bis 1000 liefern beide dasselbe (mit Java geprüft) (2 BE).

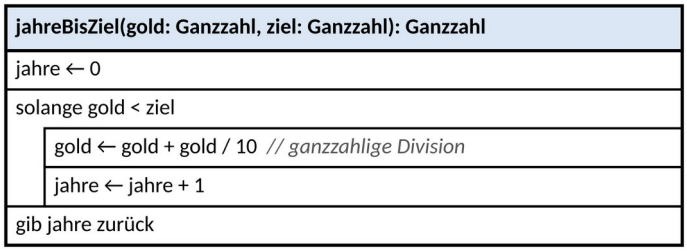
c) Ausgabe (2 BE): `#####` · `.#.#.` · `..#..` (eine Zeile je z; # genau dann, wenn z die Spaltennummer s teilt).

Aufgabe 6 – Bauplan und Code (12 BE)

```
public static int anzahlTeiler(int n) {
    int anzahl = 0;
    for (int t = 1; t <= n; t++) {
        if (n % t == 0) {
            anzahl = anzahl + 1;
        }
    }
    return anzahl;
}
```

a) 3 BE: Kopf mit Rückgabetyp int (1), Zählschleife 1 bis **einschließlich** n (1), Bedingung `n % t == 0` mit `==` und Rückgabe (1). **b)** `anzahlTeiler(12)` = **6** (1, 2, 3, 4, 6, 12), `anzahlTeiler(7)` = **2**. Genau die **Primzahlen** haben zwei Teiler (1 und sich selbst) (2 BE).

c) Struktogramm (3 BE):



d) gold nach jedem Jahr: 110 → 121 → 133 → 146 → 160 (≥ 150) → Rückgabe **5** (2 BE).

e) Für alle gold **< 10** (0 bis 9 und alle negativen Werte): `gold / 10` ist dann 0 (bzw. bei negativem gold ≤ 0), gold wächst nie, die Bedingung `gold < ziel` bleibt immer wahr → Endlosschleife. Ab gold = 10 wächst gold in jedem Jahr um mindestens 1 (2 BE).

Aufgabe 7 – Level 0.2

Musterlösungen und häufige Fehler: `AQ_W0_Akademie_LSG.txt`, Klasse Werkstatt. **c)** Ausgabe: „Quersumme von 2026: 10 – und jahr ist danach immer noch 2026“. quersumme verändert nur ihren Parameter n, eine **Kopie** von jahr (Vorgriff auf DS 4–5: Wertübergabe). Häufig rot: `basis_istGerade_negativ` (`n % 2 == 1`), `basis_kistenBenoetigt_passtGenau` (immer + 1), `erweiterung_durchschnitt_kommazahl` (Casting-Falle), `experte_istPrimzahl_quadratzahlen` (`t * t < n` statt `<=`).

Aufgabe 8 – Bug-Hunt (8 BE)

Methode	Zeile	Fehler	Korrektur
A	2	Für negative ungerade Zahlen ist der Rest −1, nicht 1	`return n % 2 != 0;`

Methode	Zeile	Fehler	Korrektur
B	4–6	i wird nie erhöht → Endlosschleife	nach Zeile 5: `i++;`
C	2	`=` ist eine Zuweisung, keine Bedingung (Compilerfehler: int ist kein boolean)	`if (gold == 0) {`
D	2	`||`: jede Zahl ist > 12 oder < 18 → immer true	`return alter > 12 && alter < 18;`

b) Nur C meldet der Compiler. B fällt beim Ausführen auf: Das Programm **hängt** (keine Ausgabe, keine Rückgabe). A und D laufen ohne Meldung, liefern aber falsche Ergebnisse – solche Fehler finden nur Tests (2 BE). **c)** z. B. `istUngerade(-3)` liefert false (richtig: true); `istJugendlich(30)` liefert true (richtig: false) (2 BE).

Für Schnelle – Zahlen mit Eigenschaften

Perfekte Zahlen unter 10 000: **6, 28, 496, 8128**. Armstrong-Zahlen: **153, 370, 371, 407**. Zauberfolge: Rekordhalter bis 1000 ist **871** mit 178 Schritten; n = 27 braucht 111 Schritte. Gute Lösungen nutzen eine Hilfsmethode `teilersumme(n: Ganzzahl): Ganzzahl` bzw. `Werkstatt.potenz` und `Werkstatt.quersumme`-artige Ziffernzerlegung.